

Rapport Technique : Optimisation de Chargement par Branch-and-Bound

TP Recherche Opérationnelle

Réalisé par : Tasnim BAGHA

Groupe : 1

Niveau : 4ème année ingénieur

Date : Janvier 2026

Dépôt GitHub

Le code source complet, les instances de test et les résultats sont disponibles sur GitHub :

<https://github.com/BaghaTasnim/knapsack-branch-and-bound.git>

1. Introduction

Dans le cadre de ce TP, j'ai travaillé sur un problème classique d'optimisation combinatoire : le problème du sac-à-dos 0-1. L'objectif était d'implémenter une méthode exacte appelée Branch-and-Bound pour résoudre ce type de problème.

Le contexte C'est celui d'une entreprise de logistique qui doit optimiser le chargement de conteneurs. Chaque conteneur a une capacité de poids limitée, et nous disposons d'articles ayant chacun un poids et une valeur (représentant le profit généré). Le but est de maximiser la valeur totale transportée sans dépasser la capacité du conteneur.

Ce problème est NP-difficile, ce qui signifie qu'il n'existe pas d'algorithme polynomial connu pour le résoudre de manière optimale. C'est pourquoi j'ai utilisé la méthode Branch-and-Bound qui, bien qu'ayant une complexité exponentielle dans le pire des cas, permet d'obtenir la solution optimale en élaguant intelligemment l'espace de recherche.

2. Implémentation

2.1 Architecture du code

J'ai structuré mon code en plusieurs modules pour faciliter la maintenance et la réutilisation :

- **data_reader.py** : Gère la lecture des fichiers d'instances
- **greedy_algorithms.py** : Contient les algorithmes gloutons (fractionnaire et entier)

- **branch_and_bound.py** : Implémente l'algorithme principal B&B
- **main.py** : Programme principal avec interface interactive

Cette organisation modulaire m'a permis de tester chaque composant séparément avant de les intégrer.

2.2 Structures de données

Pour représenter un nœud de l'arbre de recherche, j'ai créé une classe `Node` qui stocke :

- Le niveau dans l'arbre (nombre de décisions prises)
- La valeur et le poids courants
- La borne supérieure calculée
- La solution partielle sous forme de liste

J'ai également utilisé une classe `KnapsackInstance` pour encapsuler les données d'une instance (capacité, poids, valeurs).

2.3 Algorithmes implémentés

Algorithme glouton fractionnaire

Cet algorithme résout la relaxation linéaire du problème en autorisant les fractions d'articles. Je l'ai utilisé pour calculer les bornes supérieures. Le principe est simple : on trie les articles par ratio valeur/poids décroissant, puis on les prend dans cet ordre jusqu'à remplir le sac, en acceptant de ne prendre qu'une fraction du dernier article si nécessaire.

Algorithme glouton entier

Version entière de l'algorithme précédent, utilisée pour obtenir rapidement une solution initiale (borne inférieure). Cette heuristique permet d'initialiser l'algorithme B&B avec une bonne solution de départ, ce qui améliore l'élagage.

Branch-and-Bound

L'algorithme principal fonctionne selon le schéma suivant :

1. Tri préalable des articles par ratio v/w décroissant (optionnel)
2. Initialisation avec la solution gloutonne entière

3. Exploration en profondeur d'abord (DFS) de l'arbre
4. À chaque nœud, calcul de la borne supérieure
5. Élagage si la borne est inférieure à la meilleure solution connue
6. Branchement : essayer de prendre l'article (branche gauche) puis de ne pas le prendre (branche droite)

J'ai choisi une stratégie DFS avec priorité à la branche gauche car elle permet de trouver rapidement des solutions complètes et donc d'améliorer l'élagage.

3. Résultats expérimentaux

J'ai testé mon implémentation sur 7 instances de tailles différentes (de 3 à 30 articles). Voici un tableau récapitulatif des résultats obtenus avec le tri activé :

Instance	n	Capacité	Valeur opt.	Nœuds explorés	Nœuds élagués	Temps (s)
instance_1.txt	3	50	220	8	4	0.0000
instance_2.txt	10	165	309	5	5	0.0000
instance_3.txt	15	750	1458	103	35	0.0000
instance_4.txt	20	400	4152	2174	532	0.0166
instance_5.txt	25	500	1790	47	19	0.0044
instance_6.txt	30	600	2427	44	23	0.0006
instance_7.txt	22	450	966	13390	3088	0.0556

On observe que les temps d'exécution restent très faibles pour la plupart des instances (< 0.02s). Seule l'instance 7 pose problème avec plus de 13 000 nœuds explorés. Cette instance est particulièrement difficile car tous les articles ont des ratios v/w très proches (entre 1.8 et 2.5), ce qui rend l'élagage moins efficace.

Les graphiques générés montrent clairement la corrélation entre le nombre de nœuds explorés et le temps d'exécution. On remarque aussi que certaines instances de grande taille (comme instance_6 avec 30 articles) se résolvent très rapidement grâce à un élagage efficace.

4. Analyse

4.1 Impact du tri des articles

J'ai comparé les performances avec et sans tri préalable par ratio v/w . Les résultats sont intéressants :

- **Instance 3** : Le tri réduit le nombre de nœuds de 89% (969 → 103)
- **Instance 5** : Réduction de 29% (66 → 47)
- **Instance 6** : Réduction de 60% (111 → 44)

Cependant, pour certaines instances, le tri dégrade les performances :

- **Instance 4** : Sans tri explore seulement 110 nœuds contre 2174 avec tri
- **Instance 7** : Sans tri trouve la solution en 1 nœud contre 13390 avec tri

Ce phénomène m'a surpris au début, mais j'ai compris que lorsque les ratios sont similaires, le tri peut forcer l'algorithme à explorer des branches non prometteuses en profondeur avant de découvrir la solution optimale. Dans certains cas, l'ordre "naturel" des données permet de tomber plus rapidement sur une bonne solution.

4.2 Qualité des bornes

La qualité de la borne supérieure est cruciale pour l'efficacité de B&B. Dans mon implémentation, j'utilise la relaxation fractionnaire qui donne une borne optimiste mais assez serrée.

Pour instance_2 par exemple, on explore seulement 5 nœuds grâce à cette borne de bonne qualité. La borne inférieure initiale obtenue par le glouton entier joue également un rôle important : plus elle est proche de l'optimum, plus l'élagage est efficace dès le début.

4.3 Améliorations possibles

Pendant le développement, j'ai pensé à plusieurs améliorations :

1. **Amélioration de la borne inférieure** : On pourrait implémenter une recherche locale (type hill climbing) à partir de la solution gloutonne pour améliorer la borne inférieure initiale.
2. **Stratégie Best-First** : Au lieu de DFS, explorer en priorité les nœuds avec les meilleures bornes supérieures pourrait réduire le nombre de nœuds pour certaines instances.
3. **Prétraitements** : Détecter les articles qui sont forcément dans la solution optimale (dominance) pour réduire la taille du problème.

5. Conclusion

Ce TP m'a permis de comprendre concrètement le fonctionnement de la méthode Branch-and-Bound. J'ai été particulièrement intéressé par l'aspect "intelligent" de l'algorithme : contrairement à une énumération exhaustive qui explorerait tous les 2^n sous-ensembles possibles, B&B évite d'explorer une grande partie de l'espace grâce à l'élagage.

Les résultats obtenus sont satisfaisants : mon implémentation trouve l'optimum pour toutes les instances testées en un temps raisonnable ($< 0.06s$ même pour $n=30$). J'ai aussi appris que le choix de la stratégie d'exploration (ordre de tri, DFS vs BFS) peut avoir un impact considérable sur les performances, et qu'il n'existe pas de stratégie universellement meilleure.

Les principales limites de mon approche sont :

- La complexité exponentielle qui peut devenir problématique pour $n > 40$
- La sensibilité à l'ordre des articles dans certains cas
- L'absence de parallélisation qui pourrait accélérer l'exploration

Pour aller plus loin, il serait intéressant d'implémenter d'autres méthodes exactes (programmation dynamique) ou des heuristiques métaheuristiques (algorithmes génétiques, recuit simulé) pour comparer leurs performances sur des instances de très grande taille.

Références

- Énoncé du TP "Optimisation de chargement et méthode Branch-and-Bound"
- Martello, S., & Toth, P. (1990). Knapsack Problems: Algorithms and Computer Implementations
- Notes de cours de Recherche Opérationnelle